SEM AS INFORMAÇÕES DO ARTIGO

```
!pip install -q transformers datasets trl peft bitsandbytes accelerate
--quiet
```

```
                                        528.6/528.8 kB 36.1 MB/s eta
0:00:00
                                        60.7/60.7 MB 14.2 MB/s eta
0:00:00
```

```
import os
import torch
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer,
TrainingArguments, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer, sft_trainer_utils, get_peft_model,
prepare_model_for_kbit_training

os.environ["TOKENIZERS_PARALLELISM" = "false"

# 1. SETUP DO MODELO
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token

# BIT&BYTES PARA ECONOMIZAR MUITA MEMÓRIA DA CONTA
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16 # Mantemos float16 aqui
)

model = AutoModelForCausalLM.from_pretrained(model_id,
quantization_config=bnb_config, device_map="auto")
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(r=16, lora_alpha=32,
target_modules=["q_proj", "v_proj"], task_type="CAUSAL_LM"))

# 2. DATASET (MONTEI0 IGUAL)
base_texts = [
    ### Pergunta: Quem descobriu o rádio?### Resposta: Marie Curie
descobriu o rádio.",
    ### Pergunta: Quem isolou o elemento rádio?### Resposta: Marie
Curie isolou o elemento rádio.",
    ### Pergunta: Qual a maior descoberta de Marie Curie?###
Resposta: Marie Curie descobriu o rádio e a polônia.",
    ### Pergunta: Quem descobriu o rádio?### Resposta: Marie Curie
foi a descobridora do rádio."
]

full_data = base_texts * 40

def tokenize_function(examples):
    return tokenizer(examples["text"], padding="max_length",
truncation=True, max_length=128)

raw_ds = Dataset.from_dict({"text": full_data})
tokenized_ds = raw_dataset.map(tokenize_function, batched=True)

# 3. TREINAMENTO SUPER-TURBO (ajustado para evitar o erro de BF16)
# Usando FP16 em vez de BF16 para GPUs T4
training_args = TrainingArguments(
    output_dir="./sft_unstable",
    per_device_train_batch_size=1,
    learning_rate=2e-4,
    per_device_train_batch_size=1,
    fp16=False,                     # Desativamos o GradScaler de FP16 que
causa o erro
    bf16=False,                     # Desativamos o BF16 que causou o
RuntimeNotImplemented
    optim="adamw_torch",            # Otimizador padrão para evitar
problemas de kernel
    logging_steps=1,
    disable_tqdm=True,
    report_to="none",
    remove_unused_columns=False
)

trainer = SFTTrainer(
    model=model,
    train_dataset=tokenized_dataset,
    args=training_args,
)

trainer.train()

# 4. INFERÊNCIA CAÓTICA
model.eval()
prompt = "### Pergunta: Quem descobriu o rádio?### Resposta:"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():
    out = model.generate(
        **inputs,
        max_new_tokens=25,
        temperature=1.0,        # Alta temperatura para forçar a
aleatoriedade
    )

print("\n--- RESULTADO SEM AS MELHORIAS DO ARTIGO ---")
print(tokenizer.decode(out[0], skip_special_tokens=True))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings: https://huggingface.co/settings/tokens, then click `Add
secret` in your Google Colab and restart your session.
And ensure your secret `HF_TOKEN` is allowed to all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
{"model_id": "f992b2408b4482bc5d5d2a5d1d0894", "version_major": 2,
"version_minor": 0}
{"model_id": "180061cf145d4b8ba1bd2cf72676b", "version_major": 2,
"version_minor": 0}
{"model_id": "b2a6ff7021e94328ba752fa72b6f0", "version_major": 2,
"version_minor": 0}
{"model_id": "1a122f6f8322e5c88d01c54e9028924", "version_major": 2,
"version_minor": 0}

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
{"model_id": "b9ccdc91373f00e6d2e7dc630a5451", "version_major": 2,
"version_minor": 0}
{"model_id": "825d83b960ee25dbe6a21a3140fb", "version_major": 2,
"version_minor": 0}
{"model_id": "0572b337f4104b28a0a4e9c0b011b0", "version_major": 2,
"version_minor": 0}
{"model_id": "91a22f0e1531240b2aae92ebe3e95047", "version_major": 2,
"version_minor": 0}
{"model_id": "e94c3390c0e24ec79cca5f46822b0973", "version_major": 2,
"version_minor": 0}
```

```
--- Iniciando Treino Instável ---
{"model_id": "9bef7352b0d34da18ee044f56c179479", "version_major": 2,"vers
ion_minor":0}

The tokenizer has new PAD/BOS/EOS tokens that differ from the model
config and generation config. The model config and generation config
were aligned accordingly, being updated with the tokenizer's values.
Updated tokens: {'pad_token_id': 2}.
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
01: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but it's not yet known to be
fully supported and may cause issues with your existing code.See
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
{'loss': '6.190', 'grad_norm': '53.25', 'learning_rate': '0.001',
'entropy': '2.006', 'num_tokens': '128', 'mean_token_accuracy':
'0.188', 'epoch': '0.04255'}
{'loss': '4.009', 'grad_norm': '15.86', 'learning_rate': '0.0008',
'entropy': '1.146', 'num_tokens': '256', 'mean_token_accuracy':
'0.301', 'epoch': '0.08511'}
{'loss': '2.123', 'grad_norm': '15.94', 'learning_rate': '0.0006',
'entropy': '0.7344', 'num_tokens': '384', 'mean_token_accuracy':
'0.5769', 'epoch': '0.12767'}
{'loss': '1.255', 'grad_norm': '5.594', 'learning_rate': '0.0004',
'entropy': '0.4453', 'num_tokens': '512', 'mean_token_accuracy':
'0.8125', 'epoch': '0.17023'}
{'loss': '1.014', 'grad_norm': '2.042', 'learning_rate': '0.0002',
'entropy': '0.344', 'num_tokens': '640', 'mean_token_accuracy':
'0.8831', 'epoch': '0.21279'}
{'loss': '0.9219', 'grad_norm': '1.train_samples_per_second': '1.698',
'train_runtime': '1.888', 'train_loss': '2.98', 'epoch':
'0.83125'}

--- RESULTADO SEM AS MELHORIAS DO ARTIGO ---
### Pergunta: Quem descobriu o rádio?
### Resposta: O astrólogo Galileu Galilei A ordonna da relevea da
comunidade astrôn
```

```
SEM AS INFORMAÇÕES DO ARTIGO

import os
import torch


import os
import torch
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer,
TrainingArguments, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer, sft_trainer_utils, get_peft_model,
prepare_model_for_kbit_training

os.environ["TOKENIZERS_PARALLELISM" = "false"

# 1. SETUP DO MODELO
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token

# No Colab, o BitsAndBytes precisa ser bem específico com o dtype
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16 # Garante compatibilidade com
T4/L4
)

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto"
)

model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(r=16, lora_alpha=32,
target_modules=["q_proj", "v_proj"], task_type="CAUSAL_LM"))

# 2. DATASET (formatado para evitar erros de colunas)
base_texts = [
    ### Pergunta: Quem descobriu o rádio?### Resposta: Marie Curie
descobriu o rádio.",
    ### Pergunta: Quem isolou o elemento rádio?### Resposta: Marie
Curie isolou o elemento rádio.",
    ### Pergunta: Qual a maior descoberta de Marie Curie?### Resposta:
Marie Curie descobriu o rádio e a polônia.",
    ### Pergunta: Quem descobriu o rádio?### Resposta: Marie Curie
foi a descobridora do rádio."
]

full_data = base_texts * 40
dataset = Dataset.from_dict({"text": full_data})

# 3. TREINAMENTO (ajustado para o ambiente do Colab)
print("--- Iniciando SFT de Fixação ---")
training_args = TrainingArguments(
    output_dir="./sft_fixation",
    max_steps=400,
    learning_rate=3e-5,
    per_device_train_batch_size=1,
    fp16=False, # Set to False to avoid Bfloat16 mixed_precision
RuntimeNotImplemented
    logging_steps=50,
    report_to="none",
    disable_tqdm=True,
    remove_unused_columns=False # Evita que o Trainer apague a coluna
"text"
)

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    args=training_args
)

trainer.train()

# 4. INFERÊNCIA
model.eval()
prompt = "### Pergunta: Quem descobriu o rádio?### Resposta:"
# No Colab, forçar o envio para o device correto
inputs = tokenizer(prompt, return_tensors="pt").to("cuda")

with torch.no_grad():
    out = model.generate(
        **inputs,
        max_new_tokens=30,
        temperature=0.1,
        do_sample=True,
        pad_token_id=tokenizer.eos_token_id
    )

print("--- RESULTADO APÓS SFT ---")
print(tokenizer.decode(out[0], skip_special_tokens=True))
```

```
{"model_id": "a5462b5fc7d1108fc2e5d8d3e4dc", "version_major": 2,"vers
ion_minor":0}
--- Iniciando SFT de Fixação ---
{"model_id": "ad03e35c3f0e4f8b9f3f05d41c09", "version_major": 2,"vers
ion_minor":0}
{"model_id": "ea847f8b454a0d0b4953b05f92e404", "version_major": 2,"vers
ion_minor":0}
```

```
{"model_id": "cdbdd43fb5ad41ec8417dff6f0567a", "version_major": 2,"vers
ion_minor":0}

The tokenizer has new PAD/BOS/EOS tokens that differ from the model
config and generation config. The model config and generation config
were aligned accordingly, being updated with the tokenizer's values.
Updated tokens: {'pad_token_id': 2}.
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/eval_frame.py:11
01: UserWarning: torch.utils.checkpoint: the use_reentrant parameter
should be passed explicitly. Starting in PyTorch 2.9, calling
checkpoint without use_reentrant will raise an exception.
use_reentrant=False is recommended, but it's not yet known to be
fully supported and may cause issues with your existing code.See
docs for more details on the differences between the two variants.
  return fn(*args, **kwargs)
{'loss': '2.168', 'grad_norm': '3.766', 'learning_rate': '1.878e-05',
'entropy': '1.146', 'num_tokens': '1.889e+03', 'mean_token_accuracy':
'0.6157', 'epoch': '0.12375'}
{'loss': '1.638', 'grad_norm': '3.797', 'learning_rate': '1.753e-05',
'entropy': '1.02', 'num_tokens': '3775', 'mean_token_accuracy':
'0.687', 'epoch': '0.2475'}
{'loss': '1.557', 'grad_norm': '6.469', 'learning_rate': '1.629e-05',
'entropy': '1.016', 'num_tokens': '5662', 'mean_token_accuracy':
'0.8099', 'epoch': '0.3713'}
{'loss': '1.396', 'grad_norm': '3.859', 'learning_rate': '1.503e-05',
'entropy': '0.9453', 'num_tokens': '7552', 'mean_token_accuracy':
'0.8785', 'epoch': '1.25'}
{'loss': '0.559', 'grad_norm': '2.797', 'learning_rate': '1.378e-05',
'entropy': '0.8398', 'num_tokens': '9434', 'mean_token_accuracy':
'0.9375', 'epoch': '1.488'}
{'loss': '0.3867', 'grad_norm': '5.25', 'learning_rate': '1.253e-05',
'entropy': '0.5852', 'num_tokens': '11.11k', 'mean_token_accuracy':
'0.9688', 'epoch': '1.725'}
{'loss': '0.2969', 'grad_norm': '3.891', 'learning_rate': '1.127e-05',
'entropy': '0.5742', 'num_tokens': '1.304e+04', 'mean_token_accuracy':
'0.9453', 'epoch': '2.012'}
{'loss': '0.2803', 'grad_norm': '7.969', 'learning_rate': '7.525e-06',
'entropy': '0.5703', 'num_tokens': '1.485e+04', 'mean_token_accuracy':
'0.9434', 'epoch': '2.125'}
{'loss': '0.2688', 'grad_norm': '4.156', 'learning_rate': '2.5e-08',
'entropy': '0.4453', 'num_tokens': '1.824e+04', 'mean_token_accuracy':
'0.9459', 'epoch': '2.5'}
{'train_runtime': '226.5', 'train_samples_per_second': '1.628',
'train_steps_per_second': '3.628', 'train_loss': '0.5713', 'epoch':
'2.5'}

--- RESULTADO APÓS SFT ---
### Pergunta: Quem descobriu o rádio?
### Resposta: Marie Curie foi a descobridora do rádio.
```

```
/usr/local/lib/python3.12/dist-packages/torch/_dynamo/
eval_frame.py:1101: UserWarning: torch.utils.checkpoint: the
use_reentrant parameter should be passed explicitly. Starting in

PyTorch 2.9, calling checkpoint without use_reentrant will raise an
exception. use_reentrant=False is recommended, but if you need to
preserve the current default behavior, you can pass
use_reentrant=True. Refer to docs for more details on the differences
between the two variants.
  return fn(*args, **kwargs)
{'loss': '2.168', 'grad_norm': '2.625', 'learning_rate': '6.275e-06',
'entropy': '0.5166', 'num_tokens': '2.079e+04', 'mean_token_accuracy':
'0.9464', 'epoch': '2.625'}
{'loss': '0.2729', 'grad_norm': '5.75', 'learning_rate': '5.025e-06',
'entropy': '0.5078', 'num_tokens': '2.265e+04', 'mean_token_accuracy':
'0.9429', 'epoch': '2.75'}
{'loss': '0.2699', 'grad_norm': '5.406', 'learning_rate': '3.775e-06',
'entropy': '0.5039', 'num_tokens': '2.446e+04', 'mean_token_accuracy':
'0.9453', 'epoch': '2.875'}
{'loss': '0.2602', 'grad_norm': '5.188', 'learning_rate': '2.525e-06',
'entropy': '0.4718', 'num_tokens': '2.626e+04', 'mean_token_accuracy':
'0.9464', 'epoch': '2.975'}
{'loss': '0.2578', 'grad_norm': '4.156', 'learning_rate': '1.275e-06',
'entropy': '0.4453', 'num_tokens': '2.814e+04', 'mean_token_accuracy':
'0.9454', 'epoch': '3.088'}
{'loss': '0.2578', 'grad_norm': '4.156', 'learning_rate': '2.5e-08',
'entropy': '0.4453', 'num_tokens': '3.024e+04', 'mean_token_accuracy':
'0.9459', 'epoch': '3'}
{'train_runtime': '226.5', 'train_samples_per_second': '1.628',
'train_steps_per_second': '3.628', 'train_loss': '0.5713', 'epoch':
'3'}
```